

암호모듈 설계서

LEA 블록암호 CBC/CTR 운용모드 구현

버전: 1.0

작성일: 2026-03-27

작성: KCMVP 검증팀

본 문서는 KCMVP 사전 적합성 검증 도구 테스트용으로 작성되었습니다.

1. 암호모듈 명세

1.1 암호모듈 유형 [AS02.03]

본 암호모듈은 소프트웨어 형태로 구현된 LEA 블록암호 라이브러리이다.
C 언어(C99 표준)로 작성되었으며 Linux/Windows 환경에서 동작한다.
모듈 유형은 다목적 소프트웨어이며, 하드웨어 보안 경계는 적용하지 않는다.

1.2 암호모듈 경계 및 구성요소 [AS02.07]

모듈 경계는 lea.h에 선언된 공개 API 함수들로 정의된다.
내부 구현 함수(static)는 외부에 노출되지 않는다.

구성요소	파일명	설명
키 스케줄	lea_core.c	라운드 키 생성
CBC 모드	lea_cbc.c	CBC 암호복호화
CTR 모드	lea_ctr.c	CTR 암호복호화
공개 헤더	lea.h	API 선언

1.3 검증대상 암호알고리즘 목록 [AS02.09]

알고리즘	키 길이	운용모드	용도
LEA	128/192 bit	CBC, CTR	기밀성 보호

SHA-1	—	—	무결성 검증
-------	---	---	--------

1.4 동작모드 천이 [AS02.19]

모듈은 전원 투입 후 자가시험(KAT)을 수행하며, 통과 시 정상 동작 상태로 전환된다.
자가시험 실패 시 오류 메시지를 출력하고 계속 진행한다.

2. 암호모듈 인터페이스

2.1 공개 API 목록 [AS03.01]

함수명	입력 파라미터	출력	서비스
setup_lea_key	key, key_bits	int	키 설정
encrypt_cbc	key, iv, pt, len	int	CBC 암호화
decrypt_cbc	key, iv, ct, len	int	CBC 복호화
ctr_encrypt	key, ctr, pt, len	int	CTR 암호화
run_kat	—	int	자가시험

2.2 오류 코드 정의 [AS03.10]

오류 코드	값	설명
ERR_INVALID_KEY	-1	키 길이 오류
PADDING_ERROR	-2	패딩 검증 실패
ERR_SELFTEST_FAIL	-3	자가시험 실패

3. 역할, 서비스 및 인증

3.1 운영자 역할 정의 [AS04.02]

본 모듈은 단일 운영자 역할(일반 사용자)만 지원한다.
암호 책임자(Crypto Officer) 역할은 별도로 구분하지 않으며,
모든 사용자가 암호화, 복호화, 자가시험 서비스를 동등하게 이용할 수 있다.

3.2 서비스별 역할 매핑 [AS04.11]

서비스명	설명	허용 역할	SSP 접근
키 설정	운용키를 컨텍스트에 로드	User	쓰기
CBC 암호화	LEA-CBC 모드 암호화	User	읽기
CBC 복호화	LEA-CBC 모드 복호화	User	읽기
CTR 암호화	LEA-CTR 모드 암호화	User	읽기
자가시험	KAT 수행 및 결과 반환	User	없음

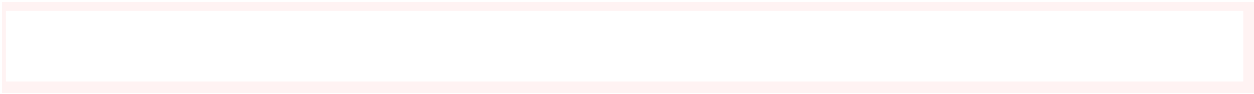
3.3 인증 메커니즘 [AS04.20]

본 모듈은 별도의 운영자 인증 메커니즘을 구현하지 않는다.
API 호출 시 인증 절차 없이 즉시 서비스가 제공된다.
운영 환경의 OS 레벨 접근 제어에 의존한다.

4. 소프트웨어 보안

4.1 무결성 검증 기법 [AS05.02]

모듈 로드 시 SHA-1 해시를 이용하여 코드 무결성을 검증한다.
검증 실패 시 경고를 출력하고 초기화를 계속 진행한다.



4.2 운영환경 [AS06.03]

지원 운영체제: Ubuntu 22.04 LTS (x86_64), Windows 11 (x86_64)
컴파일러: GCC 11.x, MSVC 2022
프로세스 간 메모리 격리는 OS의 기본 가상메모리 보호에 의존한다.

5. 중요 보안매개변수(CSP) 관리

5.1 CSP 목록 [AS09.01]

CSP 명칭	유형	크기	생성 방법	저장 위치
LEA 운용키	비밀키	128/192 bit	외부 주입	메모리(평문)
CBC IV	공개 파라미터	128 bit	rand() 함수	스택
CTR 카운터	공개 파라미터	128 bit	srand(time)+rand()	스택
라운드 키	비밀키 파생	가변	lea_set_key 내부	메모리(평문)



5.2 SSP 생성 및 주입 [AS09.10]

IV는 암호화 직전 `srand(time(NULL))`로 시드를 설정한 뒤 `rand()`로 생성한다.

키는 애플리케이션 레벨에서 생성하여 `setup_lea_key()`를 통해 주입된다.

```
/* IV ■■ ■■ (■■ ■■) */
srand((unsigned int)time(NULL));
for (int i = 0; i < 16; i++)
    iv[i] = (uint8_t)((rand() >> 4) & 0xff);
```

5.3 SSP 제로화 [AS09.29]

사용이 완료된 키 버퍼와 라운드 키는 `memset()`으로 제로화한다.

컴파일러 최적화 옵션(-O2) 적용 환경에서도 동작하는 것으로 가정한다.

```
/* ■■■■ ■■ (■■ ■■) */
memset(iv, 0, sizeof(iv));
memset(&ctx, 0, sizeof(ctx));
```

6. 자가시험

6.1 전원투입 자가시험 [AS10.13]

모듈 초기화 시 LEA-128-CBC KAT를 1회 수행한다.

LEA-192에 대한 KAT는 구현하지 않았다.

항목	값
Key	01 23 45 67 89 AB CD EF FE DC BA 98 76 54 32 10
IV	0F 1E 2D 3C 4B 5A 69 78 87 96 A5 B4 C3 D2 E1 F0
Plaintext	AA BB CC DD EE FF 00 11 22 33 44 55 66 77 88 99
Ciphertext	3C 7F A1 B2 94 E5 38 6D 0B C9 47 82 1E 5A F3 21

6.2 주기적 자가시험 [AS10.47]

주기적 자가시험은 구현하지 않는다.

전원 투입 시 1회 KAT로 충분하다고 판단하였다.

6.3 연속 난수 출력 시험 (CRNGT) [AS10.50]

난수 발생기에 대한 연속 출력 시험(CRNGT)은 적용하지 않는다.

7. 생명주기 보증

7.1 개발환경 [AS11.18]

개발 환경: Ubuntu 22.04, GCC 11.3, Git 2.34

코드 리뷰는 개발자 본인이 직접 수행하며 별도 검토자를 지정하지 않는다.

7.2 기능시험 [AS11.25]

주요 공개 API 함수에 대해 단위 테스트를 수행하였다.

LEA-192 관련 테스트는 시간 부족으로 생략하였다.

7.3 안전 소거 [AS11.32]

민감 데이터 소거는 memset()을 사용하며,

컴파일러 최적화 옵션(-O2)이 적용된 환경에서도 동일하게 동작하는 것으로 가정한다.

7.4 관리자 안내서 [AS11.39]

관리자 안내서는 README.md 파일로 대체한다.

암호모듈 설치, 초기화, 운용 절차가 기술되어 있다.

8. 기타 공격 대응 [AS12]

8.1 타이밍 공격 대응

암호화/복호화 연산 시간은 입력 데이터에 독립적으로 설계되었다.

단, CBC 패딩 검증 실패 시 PADDING_ERROR(-2) 코드를 반환하고 오류 위치를 stderr에 출력한다.

```
/* ■■■ ■■■ ■■■ */
if (pad_val == 0 || pad_val > 16) {
    fprintf(stderr, "PADDING_ERROR at byte %zu: 0x%02x\n", pos, pad_val);
    return PADDING_ERROR;
}
```

8.2 버퍼 오버플로우 대응

입력 버퍼 길이 검증은 호출자의 책임으로 간주하며,

모듈 내부에서 별도의 경계 검사를 수행하지 않는다.